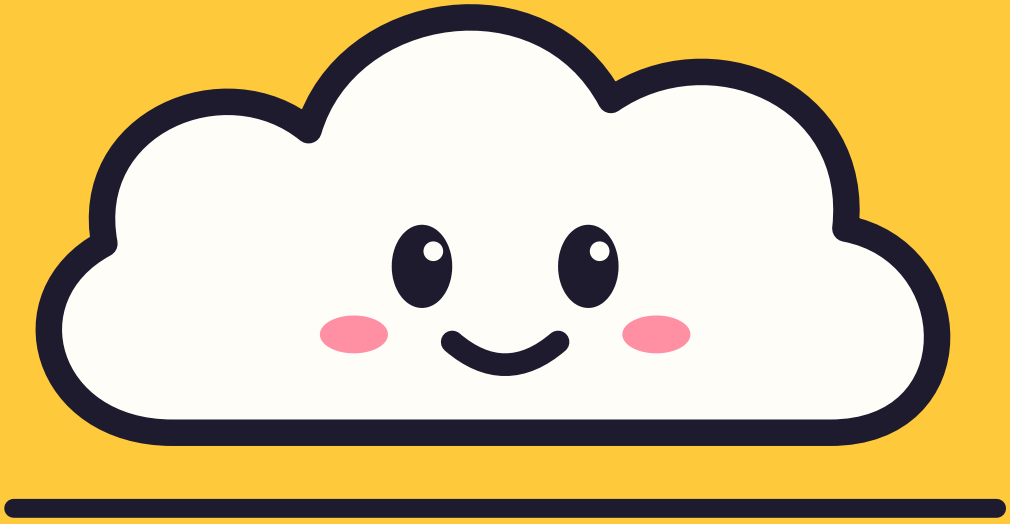
CREAT NUVEM · GUIA DO CLIENTE

Sua agência de 15 agentes dentro do Claude Code

Quem faz o quê, quando chamar cada um, como pedir, quanto custa em tokens e os erros que fazem perder tempo.

15 agentes · 6 skills · 1 fluxo em ondas



Compatível com o Claude Code. A Creat Nuvem não é um produto oficial da Anthropic e não tem vínculo com ela.

Para usar, você precisa de uma assinatura ou conta do Claude Code ativa. Cada agente consome tokens da sua conta.

Suporte do pacote: [EMAIL_SUPORTE]

O que é a agência

COMECE AQUI

O Claude Code faz quase tudo sozinho, mas faz melhor quando cada tarefa tem um dono com instruções claras. A Creat Nuvem é isso: 15 arquivos de agente, cada um com cargo, regras de qualidade e as ferramentas que deve usar. Você pede o trabalho pelo nome de quem faz.

O que vem no pacote

- `.claude/agents/`: os 15 agentes
- `.claude/skills/`: 6 skills da casa e o gerador da sede ao vivo
- `.claude/workflows/agencia.js`: o fluxo que coordena a equipe
- `CLAUDE.md`: as regras da casa
- `sede/`: a ilha 3D com os personagens

Instalar em 2 minutos

1. Copie `.claude/` e `CLAUDE.md` para a raiz do seu repositório. Se já existirem, mescle (veja o README).
2. Rode `npm install -D playwright` e `npx playwright install chromium`.
3. Abra o Claude Code e peça: `Liste os agentes disponíveis`.

Três jeitos de trabalhar

jeito	como pedir	quando usar
Um agente só	`Use o agente frontend para construir a página de preços`	tarefa de uma etapa, com dono óbvio
Uma dupla em sequência	chame o construtor e, quando terminar, o `revisor` ou o `qa` sobre o mesmo trabalho	mudança pequena que precisa de segunda opinião
A agência inteira	`rode o workflow agencia com o briefing: ...`	projeto com várias etapas: site, app, landing page, funcionalidade grande

Regra de ouro: quanto mais concreto o pedido, menos voltas. Diga o que é, para quem, o que conta como pronto e o que fica de fora. "Página de preços com 3 planos, celular primeiro, sem animação" rende mais que "faz uma página bonita".

A sede 3D

Rode `node sede/servidor.cjs` na raiz do repositório e abra `http://localhost:8000`. Você vê a ilha com os 15 personagens, cada um na sua mesa. Toque num agente para ler a ficha e copiar o comando de chamada. No modo ao vivo, a sede lê os diários dos fluxos do Claude Code na sua máquina e mostra quem está trabalhando, em quê e quantos tokens já usou.

Quem planeja e pensa

Chame estes antes de construir. Um plano ruim custa mais tokens depois do que custa agora.

Nimbus `diretor`

Diretor de Operações
Sonnet · só lê

Faz transforma o briefing num plano em ondas: tarefas, um responsável por tarefa, arquivos e critérios de aceite. Não escreve código.

Quando primeiro, em todo projeto com mais de uma etapa.

```
Use o agente diretor para planejar um site de cardápio para uma pizzaria, com pedido pelo WhatsApp, em no máximo 6 tarefas.
```

```
Use o agente diretor para ler este repositório e dizer em que ordem eu devo atacar as issues abertas.
```

Faísca `criativo`

Diretor Criativo
Opus

Faz gera 3 conceitos por metáfora, inversão e ritual, pontua e recomenda um, com o momento-assinatura escrito segundo a segundo.

Quando antes do design, quando o projeto precisa ser lembrado; ou quando algo pronto está "correto mas sem graça".

```
Use o agente criativo para propor 3 conceitos para a landing page de um estúdio de tatuagem em Curitiba.
```

```
Use o agente criativo para dizer por que a home atual parece template e sugerir um momento-assinatura.
```

Cirro `arquiteto`

Arquiteto de Software
Opus

Faz escolhe a stack mais simples que aguenta o requisito, define pastas, modelo de dados e contratos de API. Registra decisões em ADR.

Quando antes de frontend e backend trabalharem em paralelo, ou numa mudança que atravessa o sistema.

```
Use o agente arquiteto para definir a estrutura e o contrato da API de um app de lista de compras com login.
```

```
Use o agente arquiteto para escrever um ADR comparando SQLite e Postgres para este projeto.
```

Radar `pesquisador`

Pesquisador
Sonnet

Faz traz fatos com fonte: concorrentes, referências, documentação, preços. Cruza duas fontes e diz a data de cada informação.

Quando antes de decidir algo que depende de um fato que você não confirmou.

```
Use o agente pesquisador para listar 5 barbearias de São Paulo com agendamento online e o que cada site faz na primeira tela.
```

```
Use o agente pesquisador para confirmar na documentação oficial a versão atual do Three.js e o que mudou nos addons.
```

Quem constrói

O núcleo da equipe. Todos rodam em Opus e editam arquivos. Dê a cada um a lista de arquivos que ele pode tocar.

Pixel `frontend`

Engenheiro Frontend
Opus

Faz páginas, componentes e formulários, celular primeiro, com foco visível, contraste AA e tokens do projeto. Abre a página no navegador antes de entregar.

Quando para criar ou mudar qualquer tela.

```
Use o agente frontend para construir a página de preços com 3 planos, responsiva em 375, 768 e 1440 px.
```

```
Use o agente frontend para corrigir a rolagem horizontal que aparece no menu em 390 px.
```

Forja `backend`

Engenheira Backend
Opus

Faz APIs, banco, autenticação, webhooks. Valida entrada, usa consulta parametrizada, segredo só em variável de ambiente, escreve testes.

Quando sempre que há dado que sai do navegador.

```
Use o agente backend para criar a rota POST /reservas com validação, teste de sucesso e teste de erro.
```

```
Use o agente backend para receber o webhook de pagamento de forma idempotente e documentar as variáveis no .env.example.
```

Paleta `designer`

Designer de Interface
Opus

Faz identidade visual, tokens de cor, tipografia e espaçamento, layout por seção, tema claro e escuro desenhados. Revisa UX com prints.

Quando antes do frontend, ou depois dele para revisar.

```
Use o agente designer para criar o tokens.css de uma padaria de bairro, com tema claro e escuro.
```

```
Use o agente designer para revisar a página de contato e listar os problemas visuais por ordem de impacto.
```

Verbo `copywriter`

Redator e Conteúdo
Opus

Faz textos de site, microcopy, e-mails, roteiros e SEO em PT-BR. Botão diz o que acontece; falta de dado vira um aviso de PREENCHER bem visível, nunca texto de enchimento.

Quando junto com o design, antes do frontend fechar a página.

```
Use o agente copywriter para escrever os textos da home de uma clínica veterinária, com 2 opções de título.
```

```
Use o agente copywriter para reescrever as mensagens de erro do formulário dizendo o que houve e como resolver.
```

Quem dá vida

A camada que diferencia um site comum. Use com moderação: a direção de arte da casa pede uma ousadia só por projeto.

Órbita `motion-3d`

Animação e 3D
Opus

Faz cenas Three.js em estilo desenho, animações CSS e GSAP, transições e microinterações. Anima só transform e opacity; sempre com versão para movimento reduzido.

Quando 3D, mascote, transição de página, efeito de rolagem.

```
Use o agente motion-3d para criar um mascote 3D com a skill toon-3d que pula ao ser tocado.
```

```
Use o agente motion-3d para animar a entrada dos cartões em sequência, com 200 ms e prefers-reduced-motion.
```

Prisma `shader`

Creative Coder
Opus

Faz shaders GLSL, partículas, ruído, pós-processamento e arte generativa com semente. Mede o custo e tem plano B se o WebGL falhar.

Quando o efeito que um componente comum não faz.

```
Use o agente shader para criar um fundo de ruído animado em amarelo que rode a 60 fps num celular médio.
```

```
Use o agente shader para trocar o vídeo do hero por partículas e manter um gradiente CSS como alternativa.
```

Eco `som`

Designer de Som
Opus

Faz microssons e trilha ambiente sintetizados em Web Audio, sem arquivos de áudio. Nunca toca sozinho; volume baixo; botão de mudo que lembra a escolha.

Quando quando o som reforça um gesto da interface.

```
Use o agente som para criar o som de clique e de sucesso do formulário, com botão de som visível.
```

```
Use o agente som para fazer o tom subir conforme a barra de progresso do cadastro avança.
```

Antes de chamar estes três

- Defina com o `criativo` qual é o momento-assinatura. Efeito sem motivo é peso sem retorno.
- Diga o aparelho-alvo: "celular médio, 60 fps" muda as escolhas técnicas.
- Peça a versão para `prefers-reduced-motion` no próprio prompt.
- Para 3D em estilo desenho, cite a skill `toon-3d`: o kit já resolve material de 3 tons, contorno e pulo com comprimir e esticar.

Quem confere e publica

Revisor e segurança só leem e apontam; não editam, de propósito. Quem corrige é o agente que construiu.

Crivo `revisor`

Revisor de Código
Sonnet · só lê

Faz lê o diff atrás de bugs, casos de borda, duplicação e desvios das convenções. Cada achado com arquivo, linha e o cenário que quebra. Termina com veredito.

Quando depois de toda implementação.

```
Use o agente revisor para revisar o diff desta branch contra a main.
```

```
Use o agente revisor para procurar código duplicado em src/components e sugerir o que unificar.
```

Lupa `qa`

Qualidade e Testes
Opus

Faz roda build e testes, percorre os fluxos no navegador em 390 e 1440 px, testa casos de borda e escreve o relatório de bug com passos para reproduzir.

Quando antes de dizer que algo está pronto.

```
Use o agente qa para testar o fluxo de cadastro no celular e no tema escuro, incluindo campo vazio e clique duplo.
```

```
Use o agente qa para reproduzir o bug "o botão Enviar não responde no iPhone" e dizer o arquivo provável.
```

Muralha `seguranca`

Segurança
Sonnet · só lê

Faz procura segredos no código e no histórico, dependências vulneráveis, XSS, injeção, CORS aberto, cookies frouxos e dados pessoais em log (LGPD).

Quando antes de publicar qualquer coisa com formulário, login ou API.

```
Use o agente seguranca para auditar o formulário de contato e a rota que recebe os dados antes de publicar.
```

```
Use o agente seguranca para checar se algum token foi commitado no histórico do git.
```

Trilho `devops`

DevOps e Deploy
Opus

Faz scripts de build, CI no GitHub Actions com versões fixadas, deploy reproduzível e variáveis de ambiente. Não publica em produção sem a sua confirmação.

Quando para rodar com um comando e publicar.

```
Use o agente devops para configurar o GitHub Actions com lint, teste e build em cada pull request.
```

```
Use o agente devops para descobrir por que o build falha na Vercel e listar o que eu preciso configurar no painel.
```

As skills da casa

FERRAMENTAS

Uma skill é um manual que o agente carrega quando a tarefa pede. Os agentes já sabem quais usar, porque elas estão no cabeçalho de cada arquivo. Você também pode citar uma pelo nome no pedido.

skill	para quê	como usar
`ideacao`	sair da primeira ideia: mapear o mundo do cliente, gerar conceitos por metáfora, inversão e ritual, pontuar e enxertar	o `criativo` e o `diretor` usam sozinhos
`direcao-de-arte`	ficha de direção (conceito, momento-assinatura, tokens), regras de identidade e a lista de clichês proibidos	citada por designer, frontend, criativo e copywriter
`toon-3d`	3D com cara de desenho: sombra em 3 tons, contorno de nanquim, pulo com comprimir e esticar, animação a 12 poses por segundo	kit pronto em `.claude/skills/toon-3d/toon-kit.js`
`verificacao-visual`	prints em celular e desktop, claro e escuro, com erro de console, requisição quebrada, rolagem lateral e texto pequeno	`node .claude/skills/verificacao-visual/olhar.cjs index.html`
`navegador`	Chromium que fica aberto entre comandos para abrir, ver, clicar, digitar, ler, tirar print e checar o console	`node .claude/skills/navegador/navegar.cjs abrir <url> --celular`
`padrao-de-entrega`	definição de pronto: cada item marcado como ok, falhou ou não verificado, e o formato do relatório	peça: "confira com o padrao-de-entrega"
`sede-ao-vivo`	não é skill de agente: é o script que lê os diários dos fluxos e gera o estado da sede 3D	roda sozinho com `node sede/servidor.cjs`

Skills de plugins

Os arquivos dos agentes citam skills de plugins (por exemplo `frontend-design`, `impeccable`, `code-review`, `security-review`). Elas só funcionam se o plugin estiver instalado na sua conta. Sem ele, o agente segue as próprias regras e ignora a referência.

Conectores (MCP)

Quem constrói herda as ferramentas da sessão: se você tem Figma, GitHub ou outro conector ligado, o agente pode usar. O pacote não traz `.mcp.json`; o README mostra como criar o do Playwright, se quiser.

Os prints são a prova. A regra da casa é: nenhuma interface está pronta sem percorrer o fluxo no `navegador`, rodar a `verificacao-visual` e olhar os prints. Peça isso explicitamente quando chamar um agente fora do fluxo.

O fluxo em ondas

A AGÊNCIA INTEIRA

O arquivo `.claude/workflows/agencia.js` passa o trabalho de agente em agente, sem você copiar e colar entre eles. Peça: `rode o workflow agencia com o briefing: landing page para uma barbearia com agendamento online`.

1 **Conceito**
três Faíscas criam conceitos por ângulos diferentes; um júri pontua e escolhe, enxertando a melhor ideia dos outros

2 **Briefing**
Nimbus lê o repositório e monta o plano em ondas, a serviço do conceito

3 **Construção**
os especialistas de cada onda trabalham em paralelo; Crivo revisa cada entrega

4 **QA**
Lupa, Muralha e Crivo procuram problemas; outro agente confere cada achado

5 **Correções**
o responsável corrige só o que foi confirmado

6 **Entrega**
Nimbus confere os critérios de aceite e escreve o relatório

Por que ondas

Tarefas da mesma onda rodam ao mesmo tempo, então não podem editar os mesmos arquivos. Uma onda só começa quando a anterior termina. Assim o designer entrega os tokens antes de o frontend precisar deles, e ninguém sobrescreve o trabalho do outro.

Opções do fluxo

- `conceito: false` pula a etapa 1. Use em tarefa técnica, sem parte visual.
- `max_tarefas` limita o plano (padrão 10).
- Exemplo: `{ briefing: "API de reservas", conceito: false, max_tarefas: 4 }`

O RELATORIO.md

Cada agente acrescenta uma seção curta ao `RELATORIO.md` da pasta do projeto: o que fez, arquivos tocados, como verificar e o que ficou pendente. Se a sua cota acabar no meio, abra esse arquivo para saber onde o trabalho parou e retome chamando o agente certo.

Um bom briefing para o fluxo

Tem cinco partes: **o que é** (landing page, app, rota), **para quem** (barbearia de bairro, clientes no celular), **o que precisa fazer** (agendar pelo WhatsApp), **o que fica de fora** (sem login, sem pagamento) e **onde fica** (`projetos/barbearia/`). Projetos para clientes ficam em `projetos/<nome>/`, um por pasta.

O fluxo só roda numa versão do Claude Code que tenha a ferramenta de workflows. Sem ela, chame o `diretor` para ter o plano e depois chame cada agente na ordem das ondas.

Custos de tokens

ANTES DO PRIMEIRO FLUXO GRANDE

O pacote é pago uma vez. O trabalho dos agentes sai da sua conta do Claude Code, em tokens, toda vez que você usa. Este guia não traz valores em reais ou dólares, porque eles dependem do seu plano e mudam com o tempo. Ele mostra o que pesa e como medir.

O que consome tokens

- **Ler.** Cada agente começa com contexto próprio e lê de novo o que precisa: o `CLAUDE.md`, o arquivo dele, as skills e os arquivos do projeto.
- **Escrever.** Código, textos e relatórios gerados.
- **Olhar.** Prints lidos como imagem e saídas longas de comandos.
- **Repetir.** Rodadas de revisão e correção.

Do mais leve ao mais pesado

1. Um agente de leitura (revisor, pesquisador) sobre um diff pequeno.
2. Um agente construtor numa tarefa de uma tela.
3. Uma dupla construtor e QA, com prints.
4. O fluxo `agencia` completo: três conceitos, júri, plano, várias tarefas em paralelo, revisão, QA e correções.

Opus e Sonnet

Quem cria e constrói roda em Opus, o modelo mais capaz e o mais caro por token. Diretor, revisor, segurança e pesquisador rodam em Sonnet, que custa uma fração e perde pouco em tarefas de ler e julgar. Para gastar menos, troque `model: opus` por `model: sonnet` no arquivo de um agente e compare o resultado numa tarefa sua.

Como medir

- No Claude Code, o comando `/cost` mostra o gasto da sessão para quem usa a API; o `/usage` mostra o uso do plano para quem assina. Os nomes podem mudar entre versões: rode `/help` para ver os da sua.
- Na sede ao vivo (`node sede/servidor.cjs`), cada agente em trabalho mostra os tokens já usados.
- Quem usa chave de API vê o consumo por dia no painel do console da Anthropic.

Como gastar menos

- Chame um agente só quando a tarefa tem um dono óbvio.
- Use `conceito: false` e um `max_tarefas` baixo.
- Diga quais arquivos o agente pode ler.
- Rode o primeiro fluxo num projeto pequeno e meça antes de rodar num grande.

Boas práticas e erros comuns

PARA NÃO PERDER TEMPO

Boas práticas

- **Comece pelo diretor** em tudo que tem mais de uma etapa. O plano dele diz quem faz o quê e o que conta como pronto.
- **Um dono por arquivo.** Dois agentes editando o mesmo arquivo ao mesmo tempo é o jeito mais rápido de perder trabalho.
- **Critério de aceite que dá para checar:** "abre sem erro no console em 390 px", não "fica bom".
- **Revisor e QA sempre** depois de construir. São os agentes mais baratos da equipe.
- **Commit antes de cada fluxo.** Se algo sair errado, `git diff` mostra o que mudou e `git restore` desfaz.
- **Leia o RELATORIO.md** no fim. Ele lista o que ficou pendente.
- **Olhe os prints** da verificação visual com os próprios olhos.
- **Edite os agentes.** Se um deles erra sempre do mesmo jeito, acrescente uma regra no arquivo dele.

Erros comuns

- **Briefing vago.** "Faz um site" vira um plano genérico. Diga público, objetivo e o que fica de fora.
- **Pular a revisão** para economizar. O bug aparece depois, quando custa mais corrigir.
- **Rodar o fluxo completo para um ajuste de texto.** Chame o `copywriter` direto.
- **Esquecer o Playwright.** Sem ele, `navegador` e `verificacao-visual` param com a mensagem "Playwright não encontrado".
- **Sobrescrever o próprio CLAUDE.md** ao instalar. Mescle; não substitua.
- **Abrir a sede sem internet.** A cena carrega o Three.js do CDN.
- **Segredo no repositório.** Chave e token vão em variável de ambiente; o `seguranca` procura isso.
- **Achar que o agente lembra da conversa anterior.** Cada chamada começa do zero; o que importa precisa estar no pedido ou num arquivo.

Quando um agente travar: pare, leia o que ele já fez (`git diff` e `RELATORIO.md`), corte a tarefa em duas menores e chame de novo com os arquivos exatos. Repetir o mesmo pedido raramente muda o resultado.

O que a agência não faz por você

Não publica em produção sem a sua confirmação, não cria conta em serviços externos, não paga nada e não aprova o próprio trabalho. Decisão de negócio, revisão final e publicação continuam com você.

Glossário

PALAVRAS DO GUIA

Agente (subagente)

Um arquivo em `.claude/agents/` com nome, descrição, modelo e instruções. O Claude Code o chama para uma tarefa com contexto próprio.

Skill

Manual de uma tarefa (com scripts, às vezes) em `.claude/skills/<nome>/SKILL.md`. O agente carrega quando precisa.

Workflow (fluxo)

Script que coordena vários agentes. O da casa é `agencia.js`.

Onda

Grupo de tarefas que roda em paralelo, sem tocar os mesmos arquivos.

Briefing

O pedido do projeto: o que é, para quem, o que precisa fazer e o que fica de fora.

Conceito

A ideia central de um projeto em uma frase, que orienta design, texto e código.

Momento-assinatura

A única coisa que alguém vai contar para um amigo depois de usar o site.

Critério de aceite

Condição que um agente consegue checar para dizer que a tarefa terminou.

Token

Pedaço de texto que o modelo lê ou escreve. É a unidade de consumo da sua conta.

Contexto

Tudo o que o modelo tem diante de si numa chamada: instruções, arquivos lidos, conversa.

Opus e Sonnet

Modelos do Claude. Opus é o mais capaz e mais caro por token; Sonnet custa menos.

CLAUDE.md

Arquivo de regras que o Claude Code lê no início de cada sessão naquele repositório.

MCP

Protocolo que liga o Claude Code a ferramentas externas, como Figma, GitHub ou Playwright.

Playwright

Biblioteca que controla um navegador por código. Dá os olhos ao QA.

Diff

A diferença entre duas versões dos arquivos. O revisor lê o diff, não o projeto inteiro.

RELATORIO.md

Registro parcial do projeto, escrito por cada agente ao terminar a sua parte.

Sede

A página 3D com os personagens da agência. Em modo ao vivo, mostra os fluxos reais.

Tokens de design

Variáveis de cor, fonte e espaçamento (`--cor-fundo`). Não confundir com tokens de consumo.

Licença: uso pessoal e comercial permitido, inclusive em projetos de clientes. Proibido revender, redistribuir ou publicar o pacote em si. Sem garantia. Texto completo no arquivo `LICENSE`.

Suporte do pacote: [EMAIL_SUPORTE]. Conta, cobrança e limites do Claude Code são com a Anthropic.

A Creat Nuvem é compatível com o Claude Code e não é um produto oficial da Anthropic.